北京理工大學
BEIJING INSTITUTE OF TECHNOLOGY

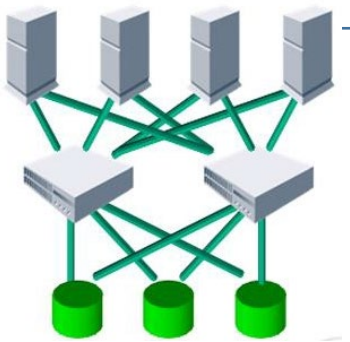
Introduction to Big Data

Big Data Processing

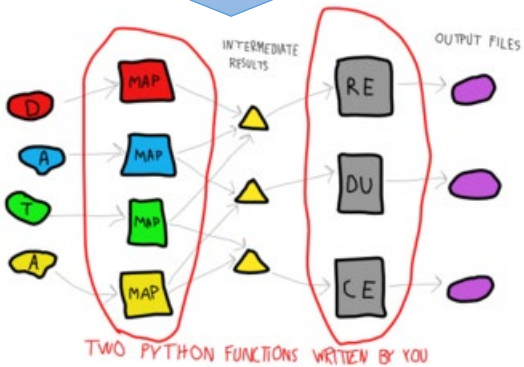
Autumn 2025

School of Computer Science & Technology

Big Data Technology



Distributed Storage



Distributed Processing

Large scale data storage

Large scale data processing

Big Data Processing

- Batch processing

  Batch processing is most often used when dealing with very large amounts of data, and/or when data sources are legacy systems.

  Batch processing works well in situations where you don't need real-time analytics results, and when it is more important to process large volumes of information than it is to get fast analytics results

All Input

All Output

Big Data Processing

- Batch processing vs. Stream processing

Under the **batch processing model**, a set of data is collected over time, then fed into an analytics system. In other words, you collect a batch of information, then send it in for processing

Under the **streaming model**, data is fed into analytics tools piece-by-piece. The processing is usually done in real time.

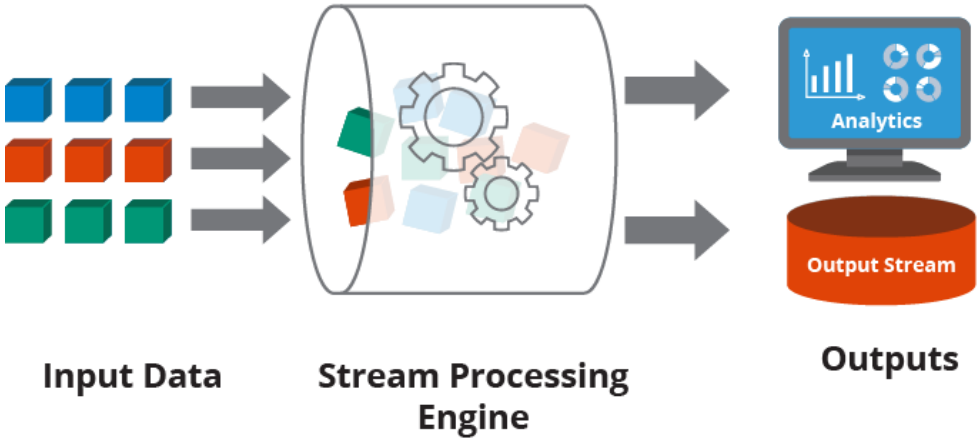
Batch processing	Stream processing
Data is collected over time	Data streams continuously.
Once data is collected, it's sent for processing	Data is processed piece-by-piece.
Batch processing is lengthy and is meant for large quantities of information that aren't time-sensitive.	Stream processing is fast and is meant for information that's needed immediately.

Big Data Processing

- Stream processing

  Stream processing is key if you want analytics results in real time. By building data streams, you can feed data into analytics tools as soon as it is generated and get near-instant analytics results.

  Stream processing is useful for tasks like fraud detection. If you stream-process transaction data, you can detect anomalies that signal fraud in real time, then stop fraudulent transactions before they are completed.



- Fraud detection
- Social media sentiment analysis
- Log monitoring
- Analyzing customer behavior

Spark Streaming

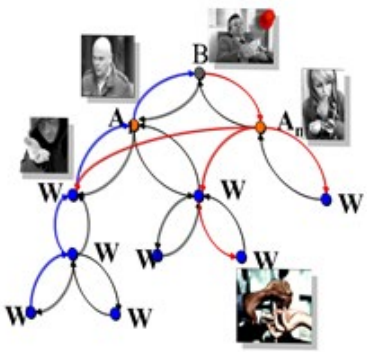
STORM

Big Data Processing

- Graph processing

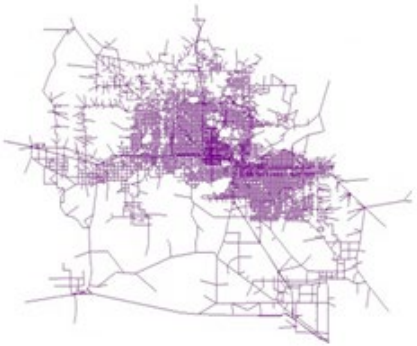
  A **graph processing** framework (GPF) is a set of tools oriented to process graphs. Graph vertices are used to model data and edges model relationships between vertices.

  Graph processing systems: Pregel、GraphX、Giraph



Social Network

Knowledge Graph



Road network

Big Data Processing

- Graph processing

  A **graph processing** framework (GPF) is a set of tools oriented to process graphs. Graph vertices are used to model data and edges model relationships between vertices.

  Graph processing systems: Pregel、GraphX、Giraph

- The scale of these graphs, in some cases billions of vertices, trillions of edges poses challenges to their efficient processing
  - Shortest Path
  - Page Rank
  - Connected Components

Hadoop

What is Hadoop

- Apache open source software framework designed for *distributed storage and processing* of *large datasets* on *large clusters* of commodity hardware
  - ***Large datasets*** → terabytes or petabytes of data
  - ***Large clusters*** → hundreds or thousands of nodes
- Hadoop is developed in Java
- Doug Cutting created Hadoop in 2005, and named it after his son's toy elephant

Hadoop is Google's Tech in Open Source

SOSP 2003

The Google File System

Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung

Google*

OSDI 2004

MapReduce: Simplified Data Processing on Large Clusters

Jeffrey Dean and Sanjay Ghemawat

jeff@google.com, sanjay@google.com

Google, Inc.

OSDI 2006

Bigtable: A Distributed Storage System for Structured Data

Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach
Mike Burrows, Tushar Chandra, Andrew Fikes, Robert E. Gruber

{fay,jeff,sanjay,wilsonh,kerr,m3b,tushar,fikes,gruber}@google.com

Google, Inc.

Motivations for Hadoop

- What were the limitations of earlier large-scale computing?
- What requirements should an alternative approach have?
- How does Hadoop address those requirements?

Early Large Scale Computing

- Historically computation was processor-bound
  - Data volume has been relatively small
  - Complicated computations are performed on that data
- Advances in computer technology has historically centered around improving the power of a single machine
  - Advances in CPUs - Moore's Law
  - Single-core limitation: single-core computing can't scale with current computing needs

Distributed Systems

- Allows developers to use multiple machines for a single task

Distributed System: Problems

- Programming on a distributed system is much more complex
  - Synchronizing data exchanges
  - Managing a finite bandwidth
  - Controlling computation timing is complicated
- Distributed systems must be designed with the expectation of failure

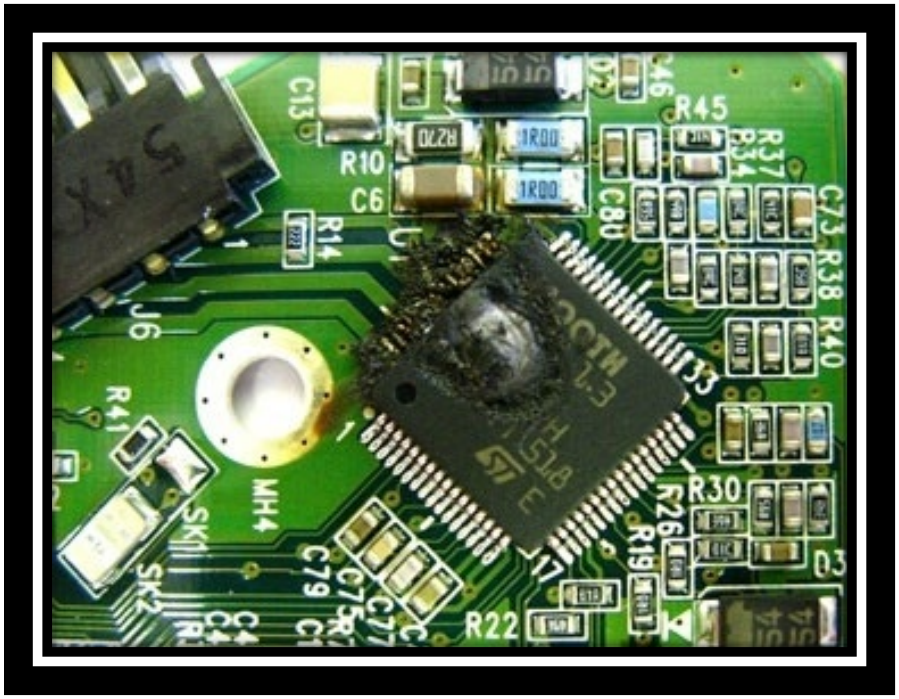
Requirements for Hadoop

- Must be easy to use
- Must support partial failure
- Must be scalable

Partial Failures

- Failure of a single component must not cause the failure of the entire system - only a degradation of the application performance

- Failure should not result in the loss of any data



Requirements for Hadoop

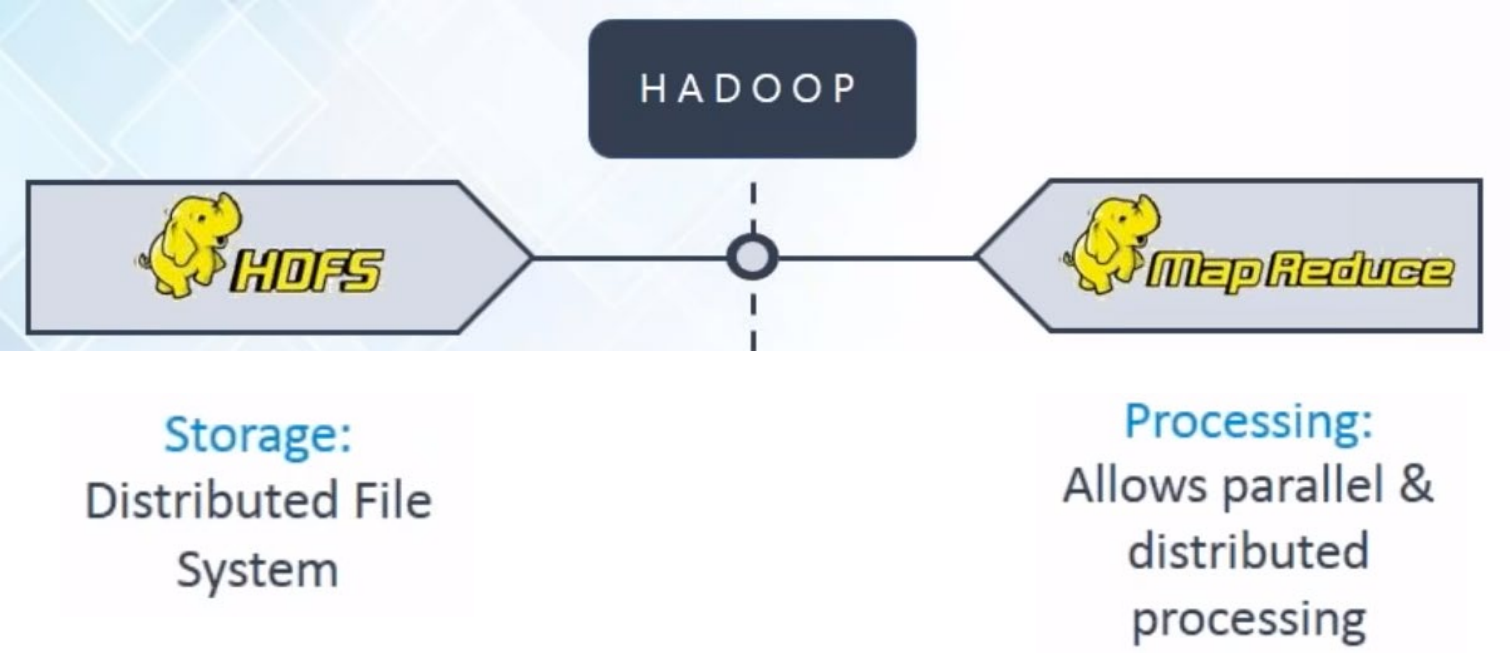
- Component Recovery
  - If a component fails, it should be able to recover without restarting the entire system
  - Component failure or recovery during a job must not affect the final output
- Scalability
  - Increasing resources should increase load capacity

Design Principles of Hadoop

- Need to process big data
- Need to parallelize computation across thousands of nodes
- Commodity hardware
  - Large number of low-end cheap machines working in parallel to solve a computing problem
- This is in contrast to Parallel DBs
  - Small number of high-end expensive machines
- Automatic parallelization & distribution
  - Hidden from the end-user
- Fault tolerance and automatic recovery
  - Nodes/tasks may fail and will recover automatically
- Clean and simple programming abstraction
  - Users only provide two functions “map” and “reduce”

Hadoop Core

- Hadoop framework that allows us to store and process large data sets in parallel and distributed fashion
  - Distributed file system (HDFS)
  - Distributed Execution engine (MapReduce)



What is MapReduce

- Hadoop is a Big Data framework designed and deployed by Apache Foundation.
- It is an open-source software utility that works in the network of computers in parallel to find solutions to Big Data and process it using the MapReduce algorithm.
- Google released a paper on MapReduce technology in December 2004. This became the genesis of the Hadoop Processing Model.
- MapReduce is a programming model that allows us to perform parallel and distributed processing on huge data sets.

What is MapReduce

MapReduce is a programming model designed for:

1. Easy parallel programming
2. Invisible management of hardware and software failures
3. Easy management of very-large-scale data

Warm Up: Word Count

- We have a large file of words, one word to a line
- Count the number of times each distinct word appears in the file
- Sample application: analyze web server logs to find popular URLs

Warm Up: Word Count (2)

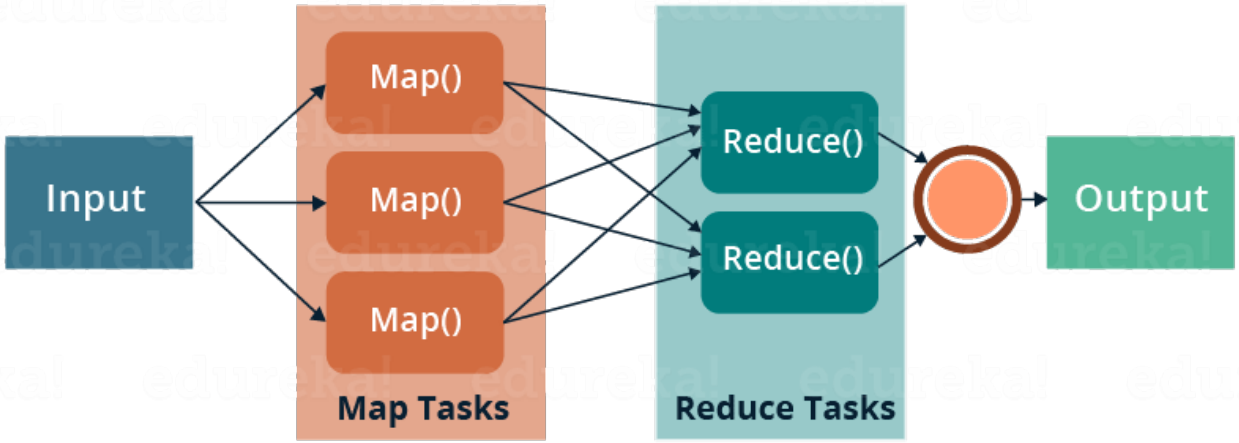
- Case 1: Entire file fits in memory
- Case 2: File too large for mem, but all <word, count> pairs fit in mem
- Case 3: File on disk, too many distinct words to fit in memory

Warm Up: Word Count (3)

- To make it slightly harder, suppose we have a large corpus of documents
- Count the number of times each distinct word occurs in the corpus
- The above captures the essence of MapReduce
  - Great thing is that it is naturally parallelizable

What is MapReduce

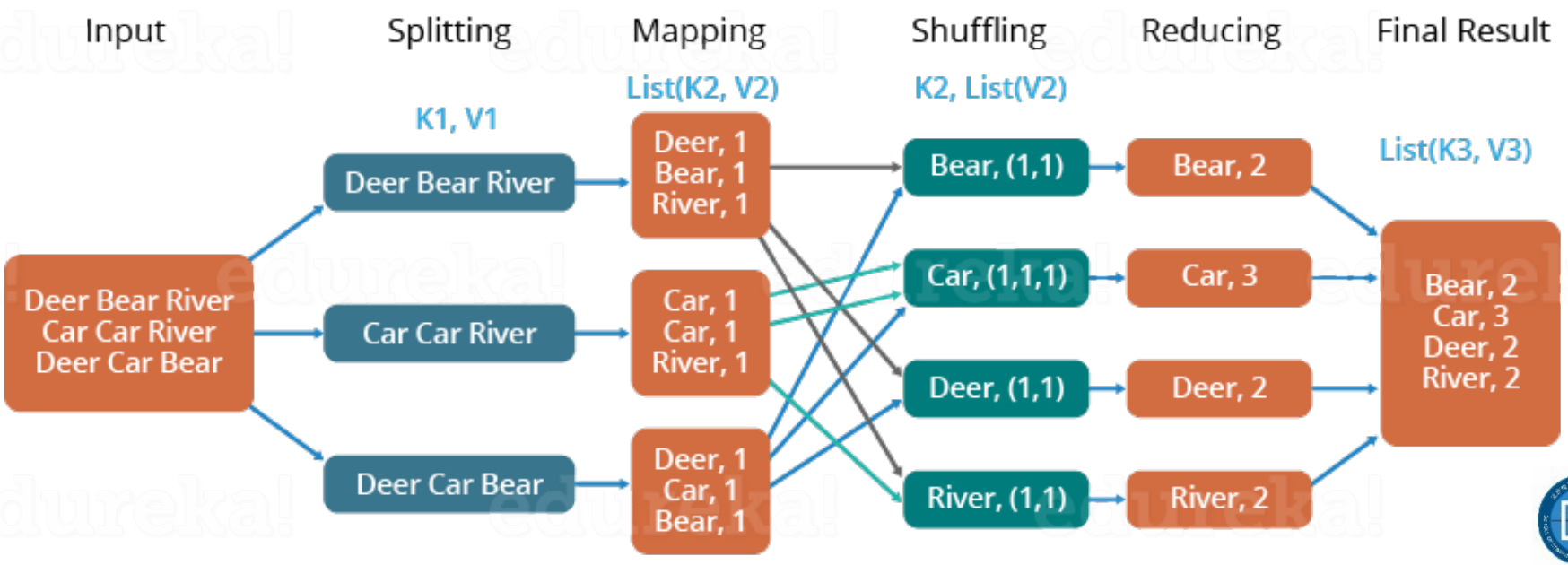
- MapReduce is a processing technique built on divide-and-conquer algorithm.
- It is made of two different tasks - Map and Reduce.
  - Map breaks different elements into tuples to perform a job
  - Reduce collects and combines the output from Map task and fetches it.



Source: https://www.edureka.co/

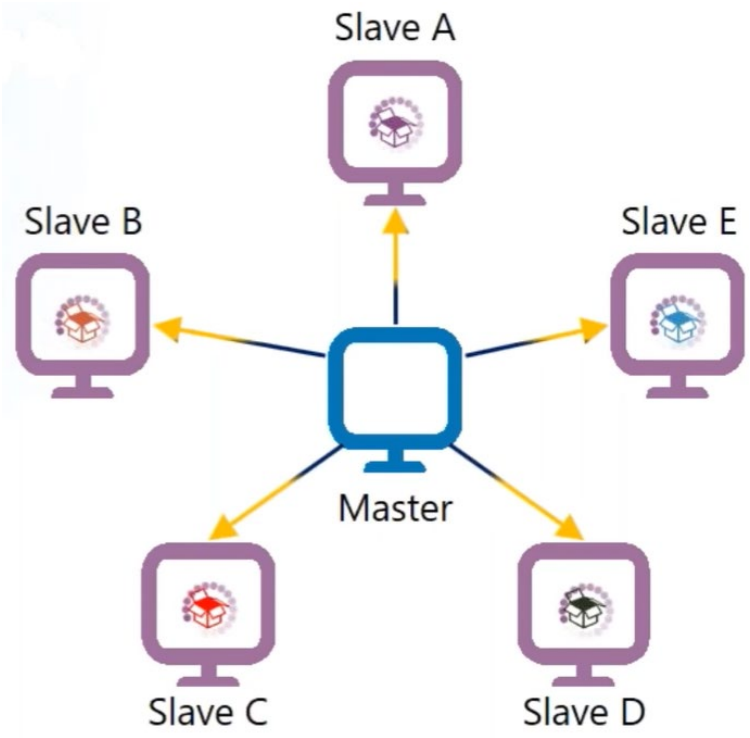
What is MapReduce

- **Map** processes a certain key-value pair and emits a certain number of key-value pairs and the **Reduce** processes values grouped by the same key and emits another set of key-value pairs as output
  - Map: $< \boldsymbol{k1}, \boldsymbol{v1} > \rightarrow \boldsymbol{list}(< \boldsymbol{k2}, \boldsymbol{v2} >)$
  - Reduce: $< \boldsymbol{k2}, \boldsymbol{list}(\boldsymbol{v2}) > \rightarrow \boldsymbol{list}(< \boldsymbol{k3}, \boldsymbol{v3} >)$



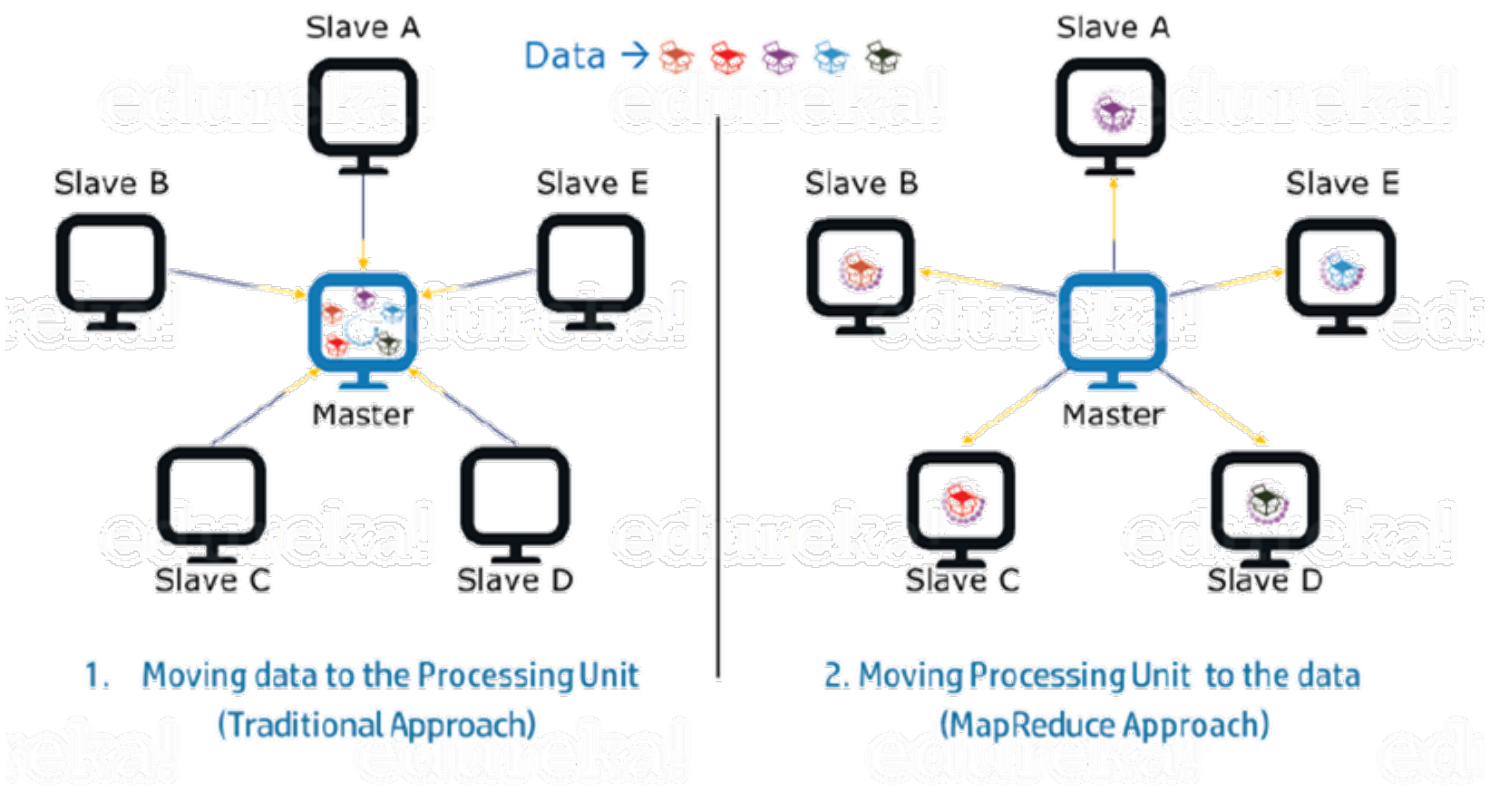
Advantages of MapReduce

- Parallel Processing: MapReduce is based on Divide and Conquer paradigm which helps us to process the data using multiple machines -- the time taken to process the data gets reduced by a tremendous amount.



Advantages of MapReduce

- Data Locality: Instead of moving data to the processing unit, MapReduce moves the processing unit to the data in the MapReduce Framework.



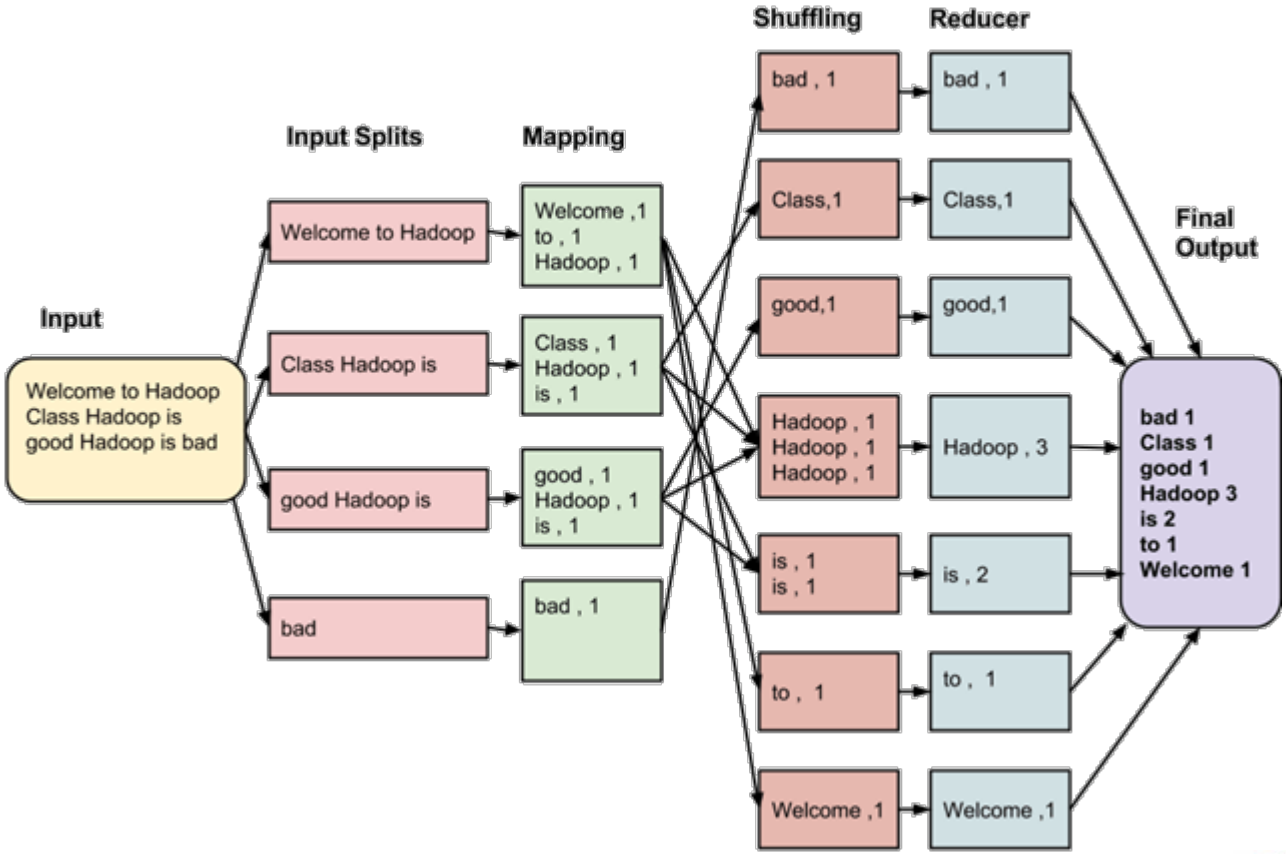
Advantages of MapReduce

- Data Locality: Instead of moving data to the processing unit, MapReduce moves the processing unit to the data in the MapReduce Framework.
  - It is very cost-effective to move processing unit to the data.
  - The processing time is reduced as all the nodes are working with their part of the data in parallel.
  - Every node gets a part of the data to process and therefore, there is no chance of a node getting overburdened.

MapReduce Phases

The data goes through the following phases in MapReduce framework:

- **Input Splits**
- **Mapping**
- **Shuffling**
- **Reducing**



Spark

What is Spark?

- Apache Spark is an open source analytics engine used for big data workloads.
- It can handle both batches as well as real-time analytics and data processing workloads.
- Apache Spark started in 2009 as a research project at the University of California, Berkeley.

amplab UC BERKELEY

ABOUT | PEOPLE | PAPERS | PROJECTS | SOFTWARE | BLOG | SPONSORS | PHOTOS

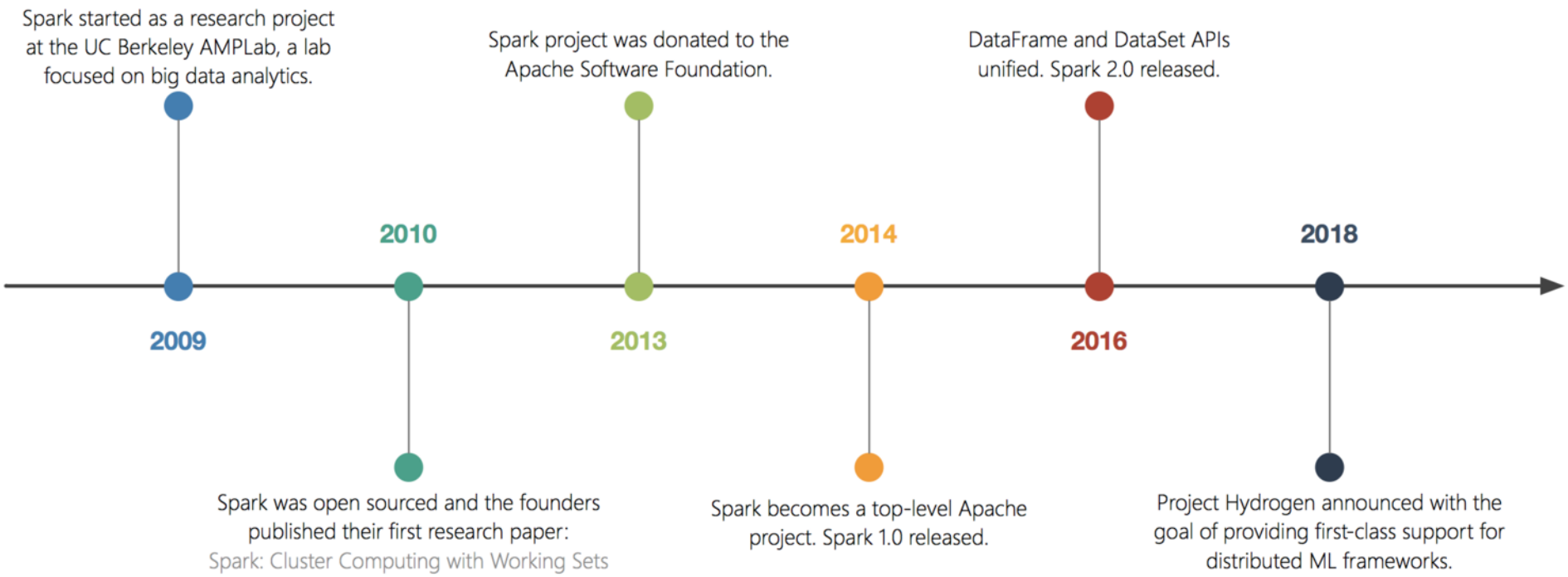
Spark – Lightning-Fast Cluster Computing

Posted on November 21, 2011 by Atchley Kattt

Spark is an open source cluster computing system that aims to make data analytics *fast* – both fast to run and fast to write. To run programs faster, Spark provides primitives for in-memory cluster computing: your job can load data into memory and query it repeatedly much quicker than with disk-based systems like Hadoop MapReduce. To make programming faster, Spark integrates into the Scala programming language, letting you manipulate distributed datasets like local collections. You can also use Spark interactively to query big data from the Scala interpreter.

What is Spark?

- Researchers were looking for a way to speed up processing jobs in Hadoop systems…



Why Spark when Hadoop is already there?

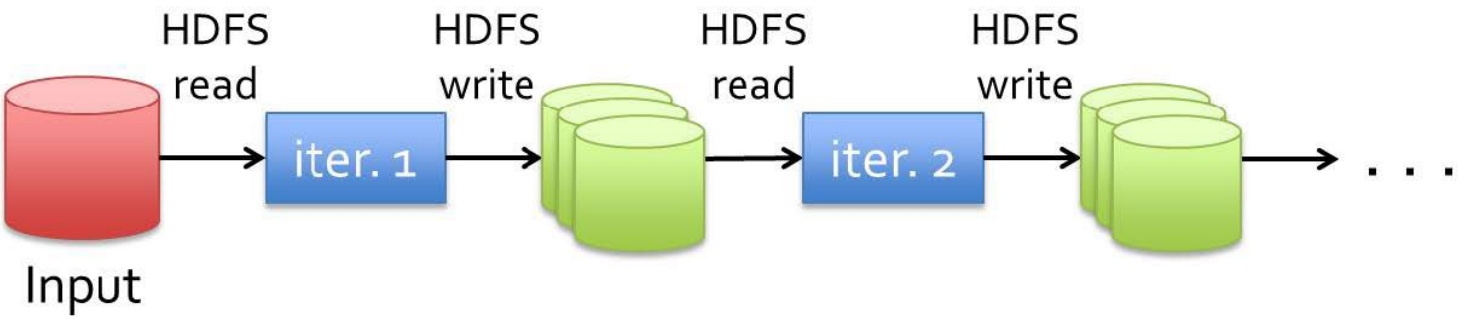
- **MapReduce**: The original scalable, general, processing engine of the Hadoop ecosystem
  - Disk-based data processing framework (HDFS files)
  - Persists intermediate results to disk
  - Data is reloaded from disk with every query → Costly I/O
  - Best for certain workloads (batch processing)
  - Costly I/O → Not appropriate for iterative or stream processing workloads

Why Spark when Hadoop is already there?

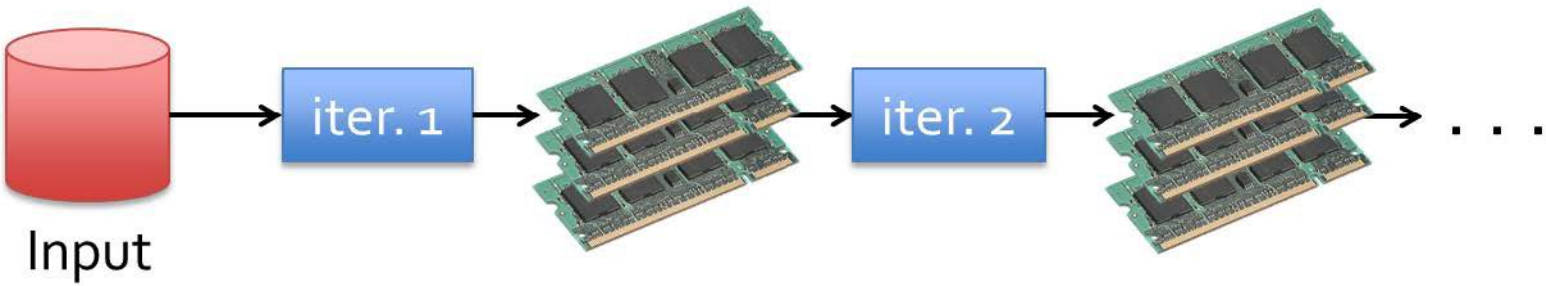
Spark: General purpose computational framework that substantially improves performance of MapReduce, but retains the basic model

- Memory based data processing framework → avoids costly I/O by keeping intermediate results in memory
- Leverages distributed memory
- Remembers operations applied to dataset (lineage)
- Data locality based computation → High Performance
- Best for both iterative (or stream processing) and batch workloads

Spark vs MapReduce



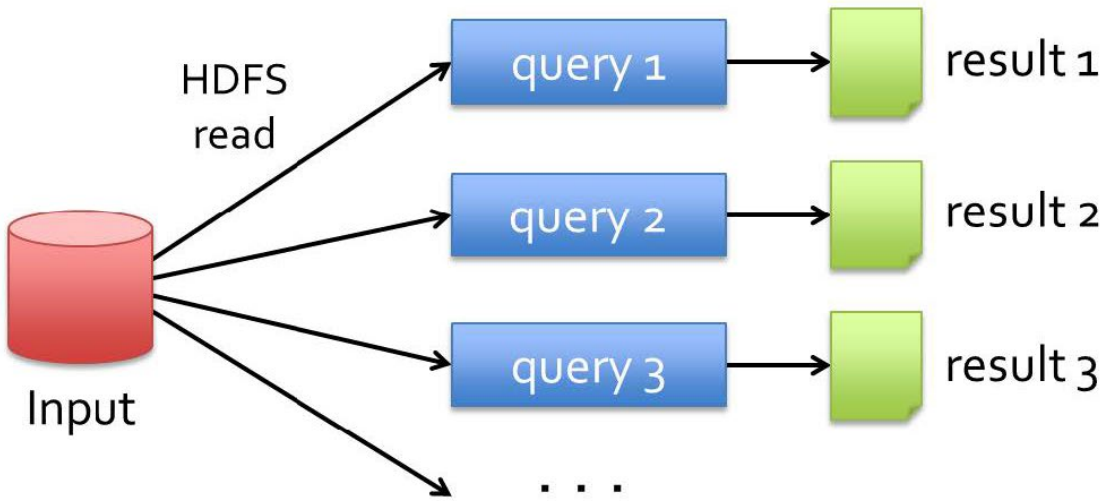
Hadoop MapReduce: Data sharing on disk



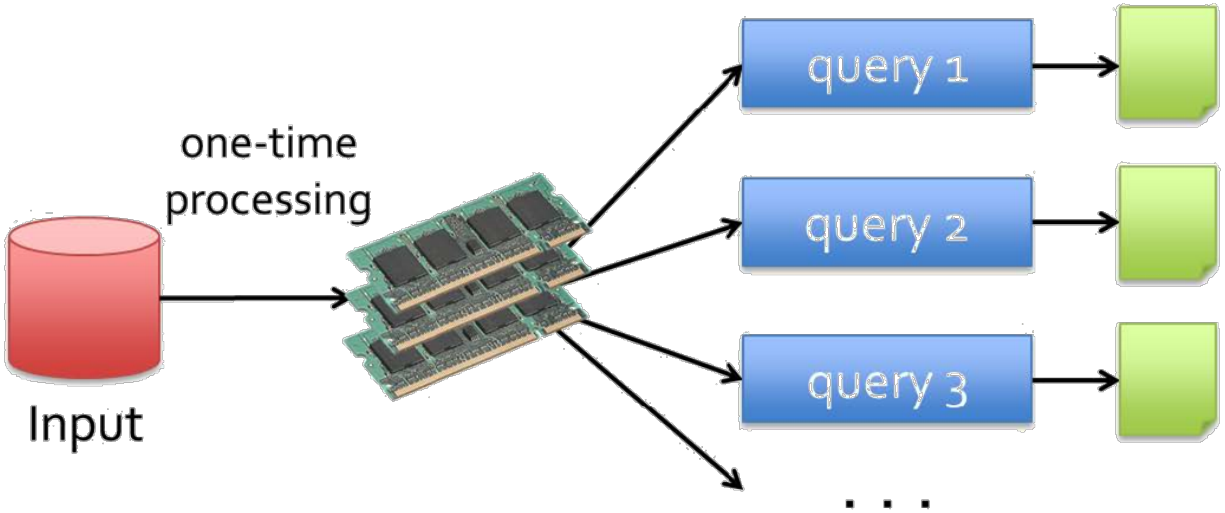
Spark: Speed up processing using memory instead of disks

The main feature of Apache Spark is its *in-memory cluster computing* that increases the processing speed of an application.

Spark vs MapReduce



Hadoop MapReduce: Data sharing on disk



Spark: Speed up processing using memory instead of disks

Spark vs MapReduce

Time to sort 100TB

2013 Record: Hadoop

2100 machines

72 minutes

2014 Record: Spark

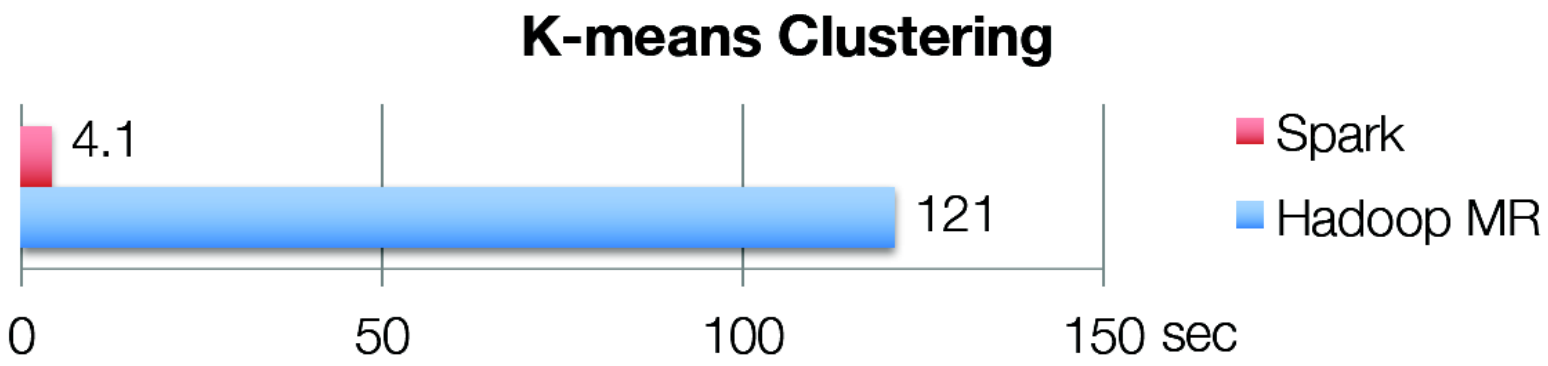
207 machines

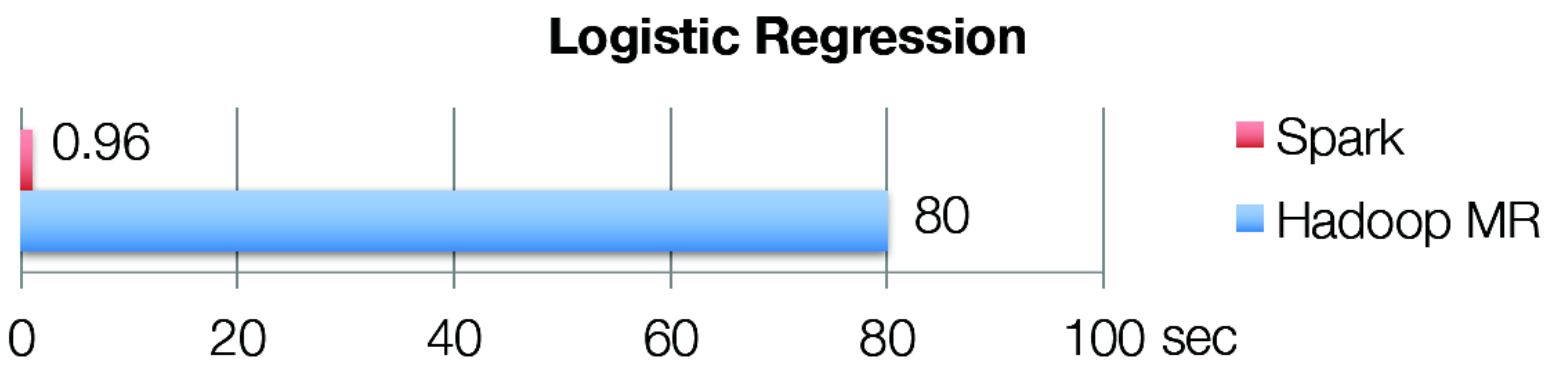
23 minutes

3X faster using 10X fewer machines

Also sorted 1PB in 4 hours

Spark vs MapReduce





Simple Spark Apps: *WordCount*

```java
public class WordCount {
  public static class TokenizerMapper
       extends Mapper<Object, Text, Text, IntWritable>{

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context
                    ) throws IOException, InterruptedException {
      StringTokenizer itr = new StringTokenizer(value.toString());
      while (itr.hasMoreTokens()) {
        word.set(itr.nextToken());
        context.write(word, one);
      }
    }
  }

  public static class IntSumReducer
       extends Reducer<Text,IntWritable,Text,IntWritable> {
    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
                       Context context
                       ) throws IOException, InterruptedException {
      int sum = 0;
      for (IntWritable val : values) {
        sum += val.get();
      }
      result.set(sum);
      context.write(key, result);
    }
  }

  public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    String[] otherArgs = new GenericOptionsParser(conf, args).getRemainingArgs();
    if (otherArgs.length < 2) {
      System.err.println("Usage: wordcount <in> [<in>...] <out>");
      System.exit(2);
    }
    Job job = new Job(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    for (int i = 0; i < otherArgs.length - 1; ++i) {
      FileInputFormat.addInputPath(job, new Path(otherArgs[i]));
    }
    FileOutputFormat.setOutputPath(job,
      new Path(otherArgs[otherArgs.length - 1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

```scala
val f = sc.textFile(inputPath)
val w = f.flatMap(l => l.split(" ")).map(word => (word, 1)).cache()
w.reduceByKey(_ + _).saveAsText(outputPath)
```

WordCount in 3 lines of Spark

WordCount in 50+ lines of Java MR

Summary